Java Day 13：内部类与匿名内部类

性能测试工程师学习计划 | Java基础进阶

一、今日学习目标

1. 理解内部类的含义，认识成员内部类和局部内部类。
2. 理解静态内部类与普通成员内部类的区别。
3. 掌握匿名内部类的基本写法。
4. 理解匿名内部类如何实现接口、重写方法。
5. 能在性能测试场景中使用匿名内部类定义一次性任务行为。

> 建议学习时间：内部类概念15分钟；成员/静态内部类15分钟；匿名内部类15分钟；实战与练习15分钟。

二、什么是内部类

定义在另一个类内部的类，称为内部类。常见形式包括成员内部类、静态内部类、局部内部类和匿名内部类。内部类适合表达与外部类关系紧密的辅助逻辑。

1. 成员内部类

```
class TestPlan {
    private String planName = "Login Performance Test";

    class Task {
        public void execute() {
            System.out.println(planName);
        }
    }
}
```

成员内部类属于外部类对象，可以直接访问外部类的实例成员，包括 `private` 成员。创建成员内部类对象时，需要先有外部类对象。

```
TestPlan plan = new TestPlan();
TestPlan.Task task = plan.new Task();
task.execute();
```

2. 静态内部类

```
class TestConfig {
    private static String environment = "TEST";
    private String owner = "QA";

    static class ReportConfig {
        public void printEnvironment() {
            System.out.println(environment);
            // Cannot directly access instance field owner
        }
    }
}
```

静态内部类使用 `static` 修饰，不依赖外部类对象即可创建。它可以直接访问外部类的静态成员，但不能直接访问外部类的实例成员。

```
TestConfig.ReportConfig config = new TestConfig.ReportConfig();
config.printEnvironment();
```

3. 局部内部类

```
class TestRunner {
    public void run() {
        class RetryPolicy {
            public boolean shouldRetry(int errorCount) {
                return errorCount < 3;
            }
        }

        RetryPolicy policy = new RetryPolicy();
        System.out.println(policy.shouldRetry(1));
    }
}
```

局部内部类定义在方法或代码块内部，作用范围局限于该方法或代码块，外部不能直接创建它的对象。

三、匿名内部类

匿名内部类没有显式的类名，通常用于只需要创建一次的实现。它可以继承一个类或实现一个接口，并在创建对象时直接提供方法实现。

1. 实现接口的匿名内部类

```
interface Task {
    void execute();
}

public class Main {
    public static void main(String[] args) {
        Task task = new Task() {
            @Override
            public void execute() {
                System.out.println("Run performance task");
            }
        };

        task.execute();
    }
}
```

`new Task()` 后面的大括号就是匿名内部类的类体。它实现了 `Task` 接口，并重写 `execute()`。

2. 继承抽象类的匿名内部类

```
abstract class PerformanceCheck {
    abstract void check();
}

public class Main {
    public static void main(String[] args) {
        PerformanceCheck check = new PerformanceCheck() {
            @Override
            void check() {
                System.out.println("Check P95 threshold");
            }
        };

        check.check();
    }
}
```

匿名内部类也可以继承抽象类，并实现其抽象方法。

四、性能测试场景：一次性执行任务

`Task` 接口定义任务规范，匿名内部类提供某次任务的具体实现。

```
interface TestTask {
    void execute();
}

public class Main {
```

```
    public static void main(String[] args) {
        TestTask loginTask = new TestTask() {
            @Override
            public void execute() {
                int responseTime = 180;
                int statusCode = 200;
                boolean success = statusCode >= 200 && statusCode < 300;

                System.out.println("Login RT: " + responseTime);
                System.out.println("Success: " + success);
            }
        };

        loginTask.execute();
    }
}
```

适用场景：一次性回调、简单任务策略、事件处理或测试工具中的临时行为。若逻辑复杂、需要复用或需要多个状态字段，建议定义具名类，避免匿名内部类过长。

五、四种内部类快速对比

类型	定义位置/方式	主要特点
成员内部类	外部类成员位置	依赖外部类对象，可访问外部类实例成员
静态内部类	`static class`	不依赖外部类对象，直接访问外部类静态成员
局部内部类	方法或代码块内部	作用范围局限在定义它的代码块
匿名内部类	创建对象时直接写类体	无显式类名，适合一次性实现

六、课后练习

练习一：成员内部类

创建 `PerformanceTestPlan` 外部类，包含 `private String planName`。定义成员内部类 `TestCase`，提供 `execute()` 方法并输出 `planName`。创建外部类对象，再创建 `TestCase` 对象并调用 `execute()`。

练习二：静态内部类

创建静态内部类 `ReportConfig`，提供 `printFormat()` 方法，输出报告格式。要求不创建外部类对象也能创建 `ReportConfig` 对象。

练习三：匿名内部类

定义 `Task` 接口，包含 `execute()` 方法。在 `Main` 中使用匿名内部类实现 `Task`，输出“开始性能测试任务”和“性能测试任务完成”。

练习四：综合实战

定义 `TestTask` 接口，包含 `execute()`。使用匿名内部类实现一个登录性能检查任务，输出响应时间、状态码和成功状态。再创建另一个匿名内部类实现查询性能检查任务。

七、思考题与验收标准

思考题：

1. 什么是成员内部类？创建成员内部类对象时为什么需要外部类对象？
2. 静态内部类与成员内部类最重要的区别是什么？
3. 局部内部类定义在哪里？作用范围是什么？
4. 匿名内部类为什么没有显式类名？适合什么场景？
5. 匿名内部类能否实现接口？能否继承抽象类？
6. 如果一个任务逻辑很复杂并且需要复用，你会选择匿名内部类还是独立类？为什么？

> 验收标准：
> - 能区分成员内部类、静态内部类、局部内部类和匿名内部类。
> - 能正确创建成员内部类和静态内部类对象。
> - 能用匿名内部类实现接口并重写方法。
> - 综合实战代码能够编译运行。
> - 思考题至少答对5道。

提交文件：`PerformanceTestPlan.java`、`Task.java`（如单独定义）、`Main.java`，以及控制台输出和思考题答案。完成后再进入 Day 14。

> 今日关键记忆：成员内部类依赖外部类对象；静态内部类不依赖外部类对象；匿名内部类适合一次性的接口实现或抽象类扩展。